

NUSphere

Your Connected Campus Marketplace & Ecosystem

NUSphere README

Team Name: NUSPHERE

Proposed Level of Achievement: Apollo 11

Core Stack: React, Vite, Django REST Framework, PostgreSQL, Cloudinary-ready media storage

Proof of Concept: https://frontend-production-8aca.up.railway.app/open-market

1. Motivation

Existing campus trade relies heavily on fragmented Telegram groups, informal friend networks, and broad external platforms such as Carousell. These channels are useful for quick messages, but they do not provide a structured marketplace experience. Posts are difficult to search after a few days, item categories are inconsistent, buyers and sellers must coordinate manually, and students who want to run small recurring businesses have few tools for inventory or order tracking.

NUSphere was motivated by the observation that NUS already has an active informal circular economy. Students moving out of hostels often need to sell furniture, electronics, bedding, and other essentials quickly. New students, especially international students, often need affordable starter items. Student entrepreneurs also sell services and goods such as baked food, 3D printing, tutoring, keyboard mods, or second-hand academic materials. The problem is not a lack of supply or demand; the problem is fragmentation, low discoverability, and weak transaction structure.

The project therefore aims to turn scattered campus commerce into a more organized student marketplace. The design does not try to remove the social and hyper-local nature of student trading. Instead, it adds just enough software structure: listings, shops, carts, order states, future notifications, and future trust mechanisms.

2. Aim

NUSphere aims to lower the barrier of entry for student sellers and improve convenience for student buyers by providing a centralized, hyper-local digital marketplace. A student should be able to post a one-off item without building a store, while a more regular seller should be able to maintain a dedicated storefront.

The long-term product direction is broader than a basic resale page. NUSphere is designed as a foundation for student micro-entrepreneurship: marketplace listings, personal stores, persistent cart behavior, order tracking, seller analytics, recommendations, and communication/notification channels.

3. User Stories

- As a student seller, I want to list handmade crafts, second-hand goods, or tutoring services easily so that I can reach the NUS community without separate marketing costs.
- As a student moving out of hostels, I want to post reusable items quickly so that I can reduce waste and recover some value before leaving.
- As a busy student buyer, I want to browse by category and search for items so that I can find useful goods between lectures.
- As an international student, I want to find affordable starter items such as bedding, kettles, and dorm essentials so that I can settle into campus life quickly.
- As a repeat seller, I want a personal store and product list so that I do not need to recreate separate one-off posts for every product.
- As a seller, I want pending order visibility and future analytics so that I can decide what to restock and which products are performing well.
- As a cautious buyer or seller, I want future escrow, QR completion, and chat features so that campus transactions can happen with less reliance on external contact details.

4. System Design

The design follows a conventional full-stack web application pattern. The frontend is a Vite-powered React single-page application. It owns routing, page layout, product cards, forms, cart controls, navigation and chat updates. The backend is a Django project using Django REST Framework APIView classes. It owns authentication, permissions, inventory transitions, order creation, cart mutation, store creation, and media handling. PostgreSQL stores relational state such as users, listings, unit-level inventory, orders, shops, products, carts and chats.

4.1 Architecture Diagram

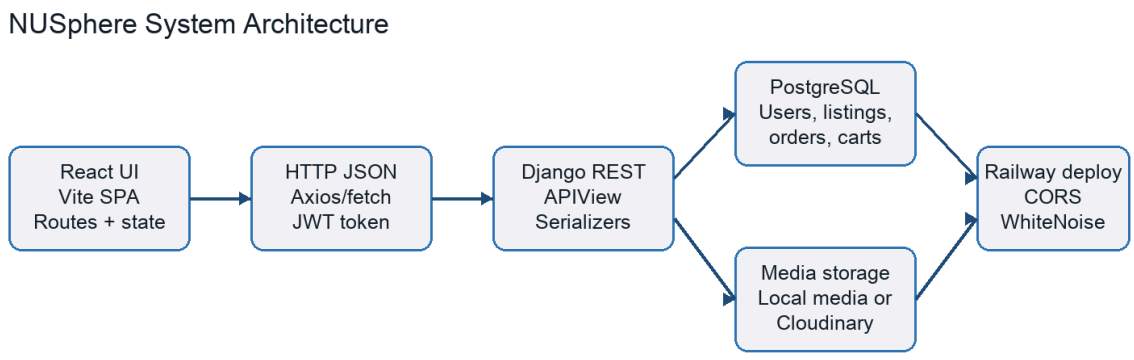


Figure 1. System architecture linking React, API requests, Django REST views, PostgreSQL, media storage, and deployment support.

4.2.1 Seller's Usage Flow

4.2.1.1 Pre-login

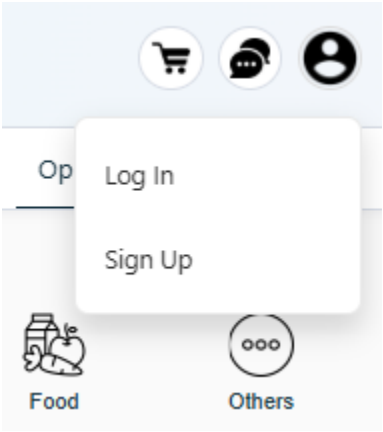


Figure 2. Pre-login tab.

1. As a seller, before you can add listings or set up your own store, you need to log in.
2. To log in, click on the profile icon to the top right of the page, and click log in/sign up to continue.

4.2.1.2 Adding own listings

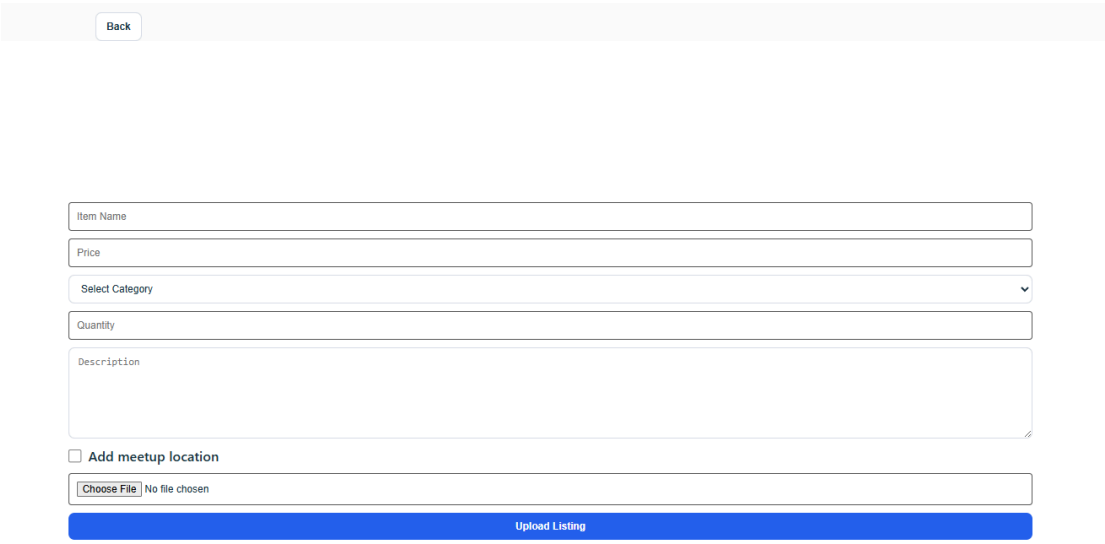


Figure 3. Example of creating a listing.

1. To add your own listings, click on your own profile picture/ the profile icon after logging in and click on "My Listings".
2. After which, click on the dotted box that states "Create New Listing".

3. Fill in the form accordingly, where every field is required and price has >= 0 while quantity has to be an integer > 0.
4. Meet up locations can be included in the listing, or can be left unchecked if there are no fixed meet up locations.
5. After filling up the form, pressing upload listing will add the listing to the database where other users can view under the “Open Market” tab, but will only show up to the seller under the “My Listings” tab.

4.2.1.3 Creating your own store

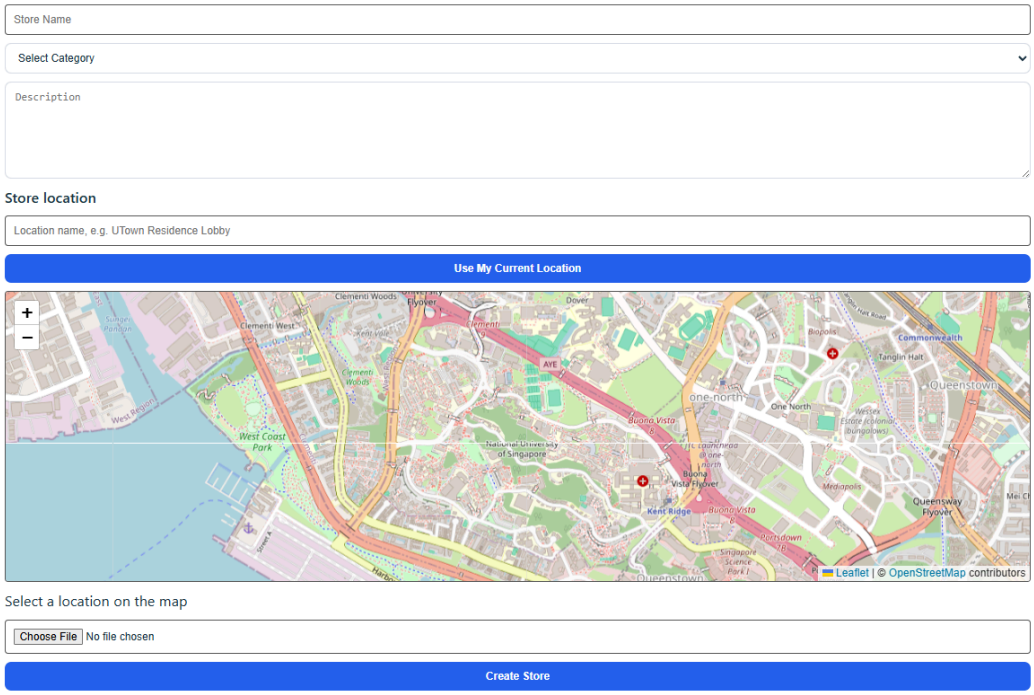


Figure 4. Example of creating a store.

1. To add your own store, click on your own profile picture/ the profile icon after logging in and click on “My Stores”.
2. After which, click on the “Add New Store” button.
3. Fill up the fields accordingly, and note that Store Name and Store Location is what will be shown to buyers to identify and locate the store.
4. After pressing create store, click into your store to add products/services and if required, you can edit the location of your store.
5. Once done, you can press the “back” button in the top left corner of the store and press the “Open Store” button such that other users can see your store under the “Stores” section.

4.2.1.4 Confirming sales

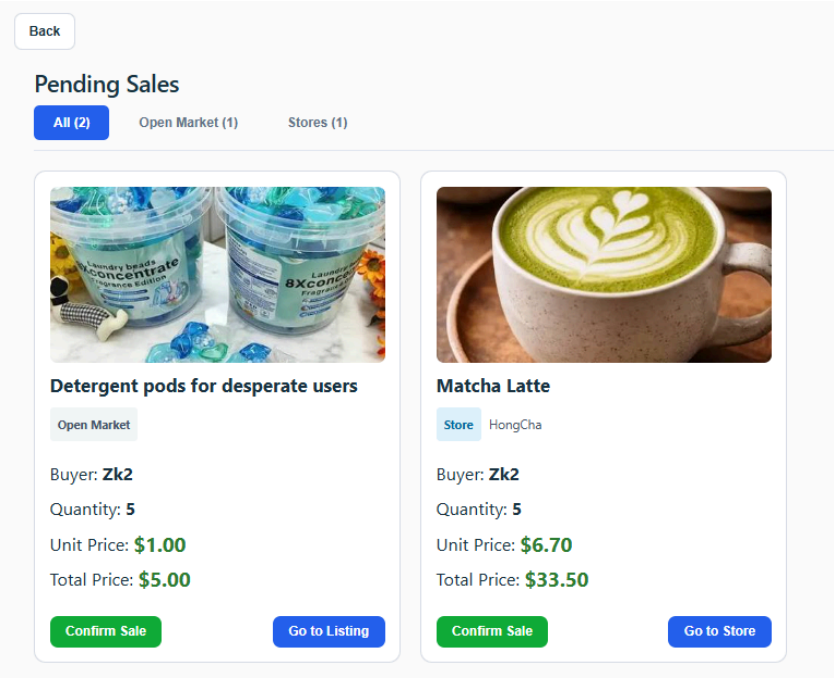


Figure 5: Example of pending sales

1. When users check out your items, you can confirm the sale under “Pending Sales”, which would signal to the buyers that you have seen their request to purchase and is ready to deal.
2. After clicking the green “Confirm Sale” button, the sale will transit from “pending” to completed at the buyer’s end, and all that's left is to meet up and deal.

4.2.1.5 Seller analytics

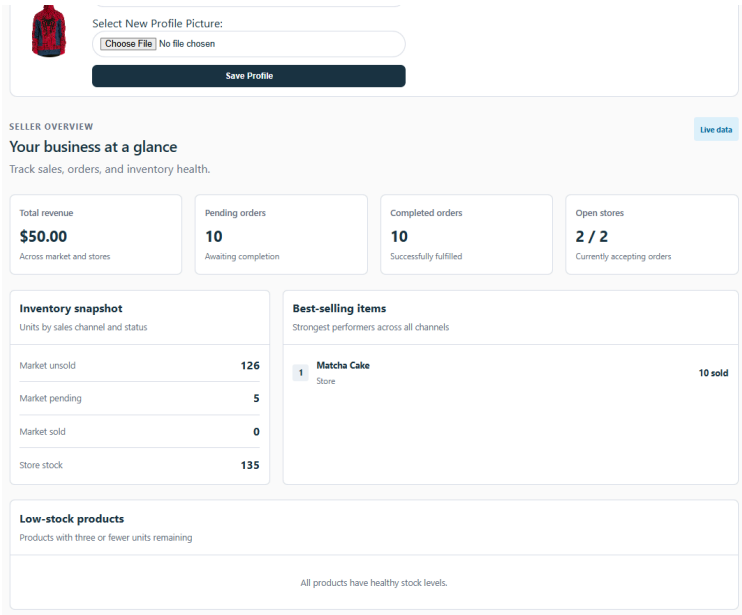


Figure 6: Seller analytics board

1. After making a few sales or putting up a few listings, sellers can start to track their sales, orders and business health through the “My Profile” tab under the profile icon.

4.2.2 Buyer’s Usage Flow

4.2.2.1 Pre-login

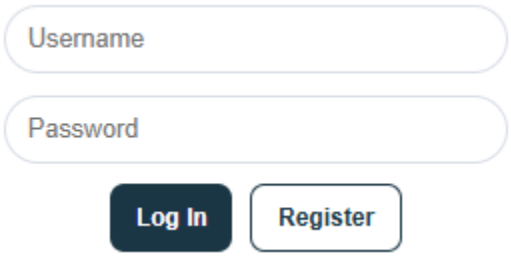


Figure 7: Example of login page

1. When users first enter the website, they will be able to view all listings and their associated details such as prices and location of meetup. After which, if they are interested in the product, they will be prompted to log in to add to cart or message sellers.
2. The login page is a classic log in or register page which users can easily follow to sign up and log in.

3. After logging in, users will have access to all features such as adding their own listings, opening their own stores, adding items to cart, checking out, and messaging sellers.

4.2.2.2 Browsing open market

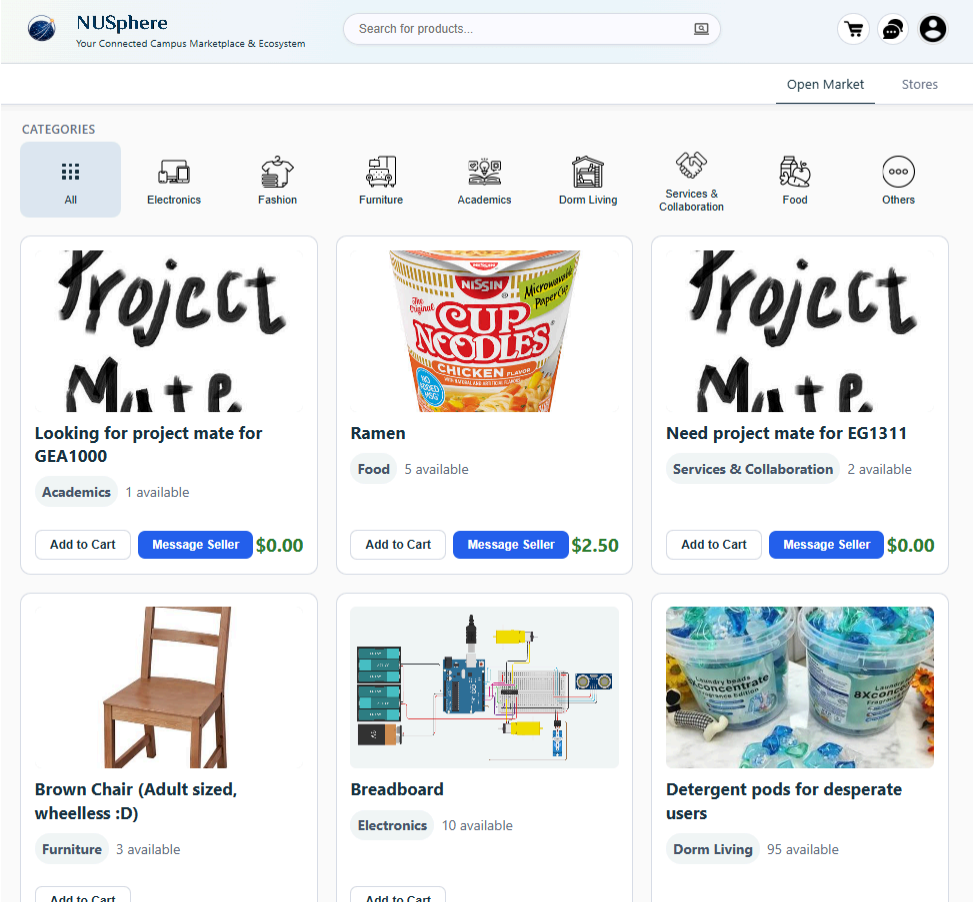


Figure 8. Example of marketplace listing.

1. After logging in, users will be redirected to the open market tab, where they will be able to see other user's listings.
2. In this tab, they can scroll through all listings, or sort listings by specific categories that they want to browse by clicking on the specified categories.
3. They can also search for products using the search bar located under the website's logo.
4. If users have any queries about specific products, they can click on the product's listing card to get a brief description, or click on "Message Seller" to ask sellers questions about the product.
5. Additionally, for products where sellers have set a pre-determined meet up location, a "Show on Google Maps" option as well as the location name would be visible under the product's description.

6. If users are interested in the product, they can click the “Add to cart” button, input an appropriate quantity and click “Confirm” to add to cart.

4.2.2.3 Browsing stores

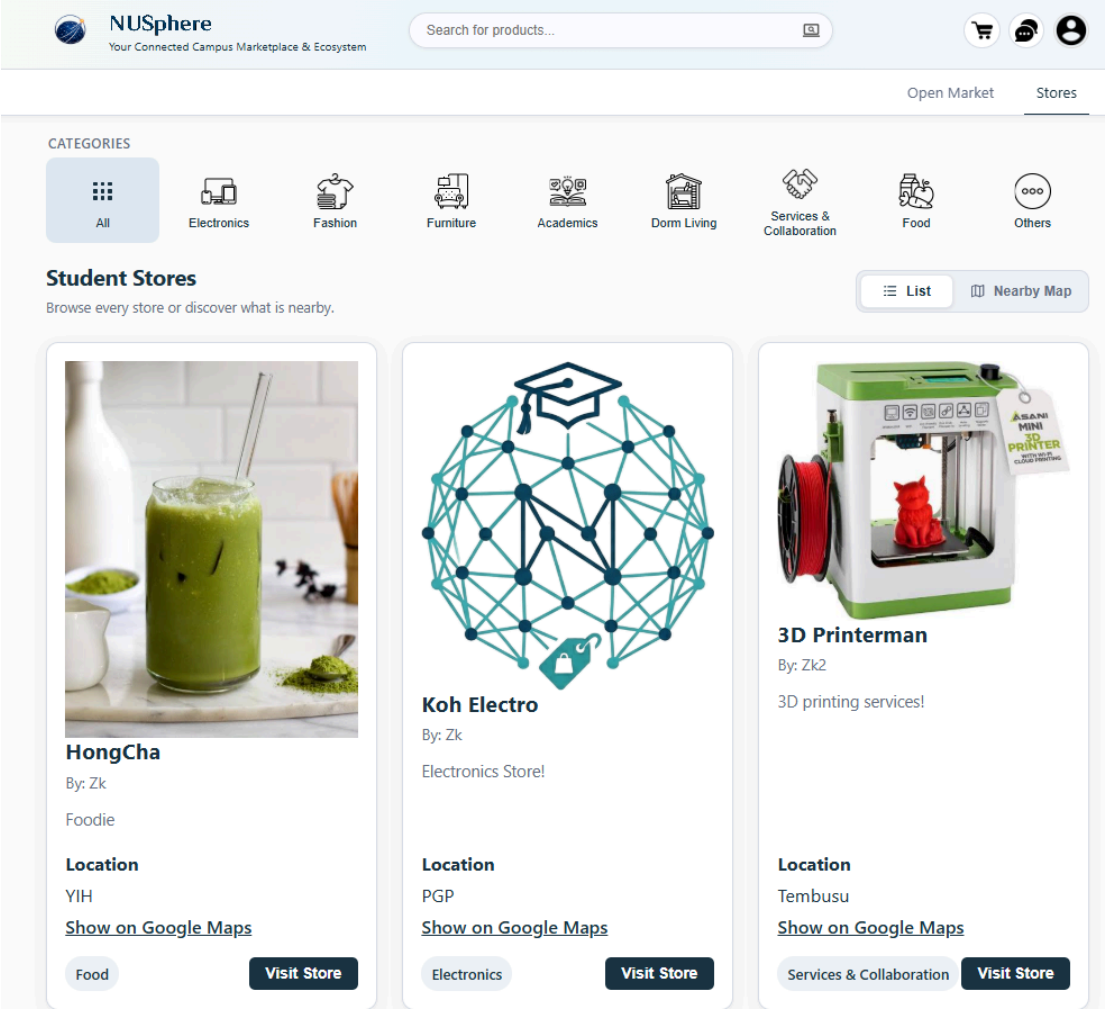


Figure 9. Example of stores.

1. To browse independent stores, users can click on the “Stores” tab in the top right corner of the page.
2. After which, users can browse through stores similar to the open market.
3. Additionally, users can also select the “Nearby Map” option instead of the default “List” option to see what stores are nearby them by either clicking on “Use My Current Location”, or clicking on a specific location on the map.
4. If users are interested in what the store is selling, they can click on “Visit Store” to enter the store and add products they are interested in to the store.

4.2.2.4 Post adding to cart

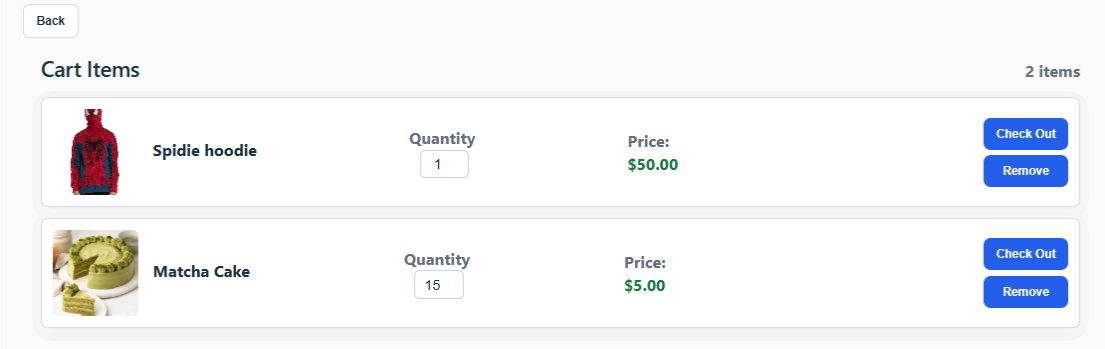


Figure 10. Example of cart.

1. After buyers are done shopping and adding to cart, they can go to the cart by clicking the cart icon in the top right corner of the website.
2. In this page, they will be able to make any final adjustments to the quantity of items they want to buy before clicking check out, which will send a pending order to the sellers.

4.2.2.5 Post check out

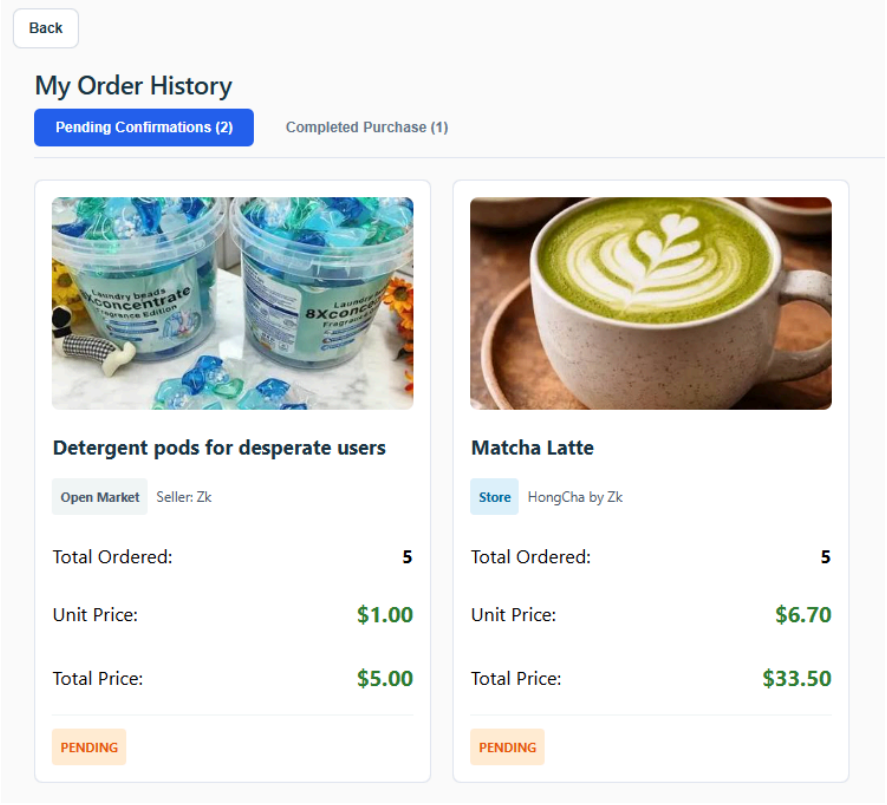


Figure 11. Example of order history.

1. After checking out, buyers can go to the “My Orders” tab under the profile icon to check their order history.

2. At this page, they can track orders which are not confirmed by sellers yet under the “Pending Confirmations” tab, or orders that are confirmed by sellers under “Confirmed Purchase” tab.

4.3 Entity Relationship Diagram (ERD)

4.3.1 Overall ERD

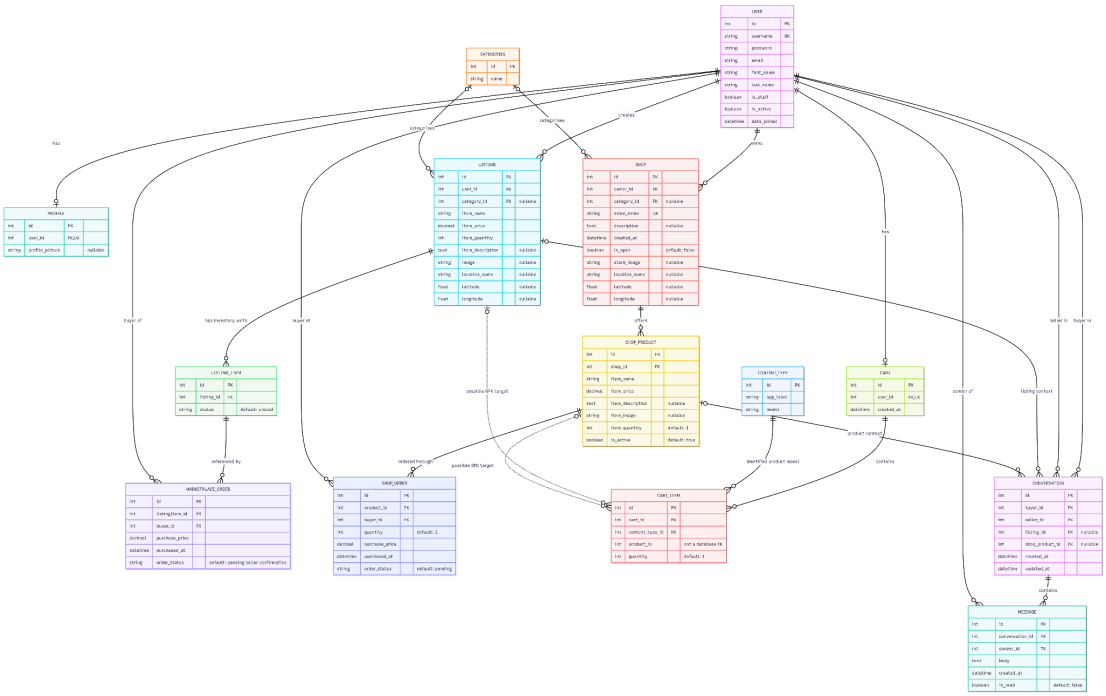


Figure 11. Overall ERD (Overall ERD.png)

Figure 11 shows the complete overview of how the relational database works in NUSphere. Each box represents a Django model, while the fields represent stored attributes.

Some notations used in the ERD include:

Notations	What it means
FK	Foreign Key: A field that references a row in another table.
PK	Primary Key: A field that uniquely identify each row in a table
UK	Unique Key: A constraint that prevents two rows from sharing the same value
Nullable	Denoting that the field is optional.

The database is divided into user identity, marketplace, store, cart and chat capabilities. The User model is the central model that is used to connect most of the models together as users create listings, own stores, make purchases, own carts and participate in conversations.

NUSphere uses Django's built-in User model to store authentication and account information. The profile model extends it by adding user-specific data such as the profile picture. A one-to-one relationship as seen from the single arm line from the User model to the Profile model shows that each user can only have one profile.

For clarity in explanation, the following paragraphs break down the remaining functionality of the database.

4.3.2 Store ERD

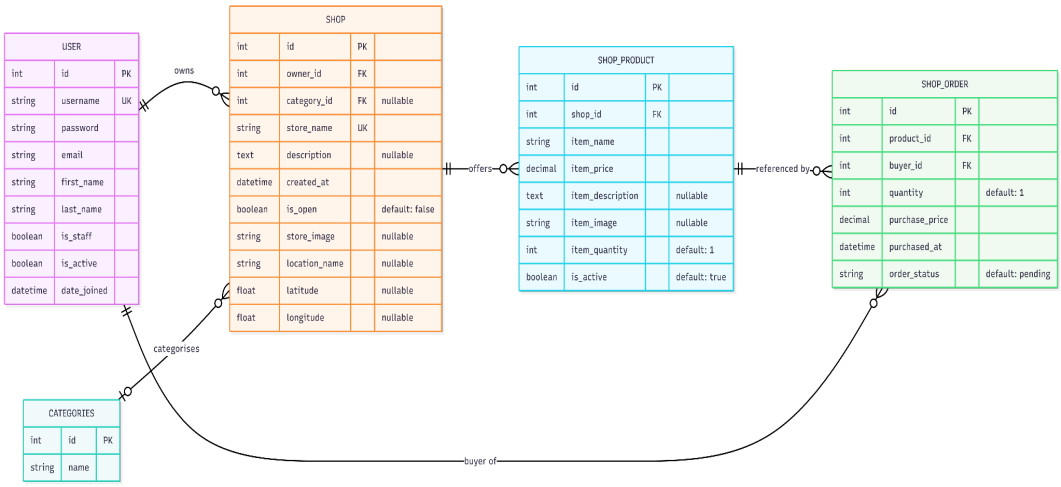


Figure 12. Store ERD (ShopProduct ERD.png)

As seen from Figure 12, the Shop model represents a seller-owned store and stores its category, location, open/closed state and owner. As owner_id is a non-unique foreign key, one user may own multiple shops. Each shop also offers multiple Shop_Product records, which represent products that are sold in the shop.

A shop product records the name, price, description, image and quantity of the product.

When a user places an order, Shop_Order is created with reference to the respective Shop_Product and records the current price, quantity and purchased date.

The purchase_price field in Shop_Order ensures that the confirmed price remains at what was listed when buyers check out even if sellers change the price afterwards, and the purchase_status field tracks the order status from "pending" to "confirmed".

4.3.3 Listing ERD

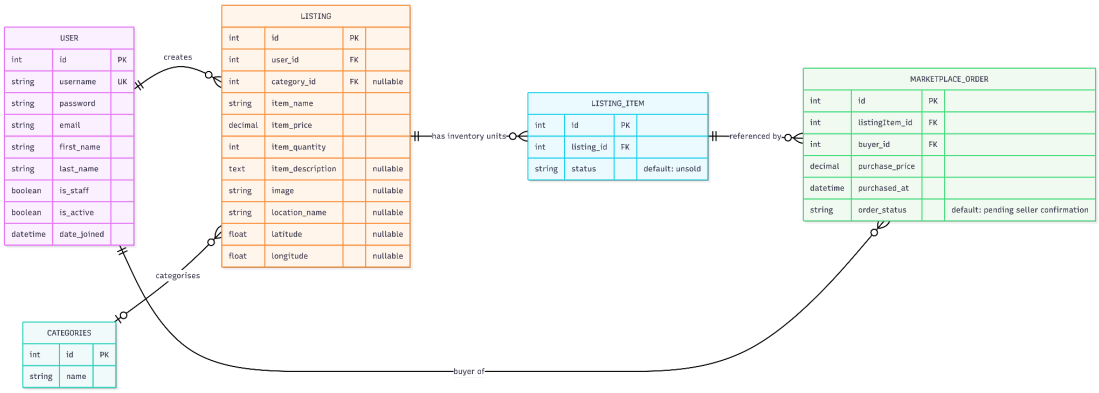


Figure 13. Marketplace ERD (ListingItem ERD.png)

As seen from Figure 13, the Listing model represents listings listed by a seller on the marketplace. It stores each listing's name, price, quantity, and category, with optional fields such as description, image, and location of the meet up which sellers can decide if they want to include. Similarly to the Shop model, a single seller can list multiple listings.

When a listing is created, one ListingItem row is created for every submitted unit, with each row representing an individual stock unit that is either unsold, pending or sold. While the total quantity of items listed is still recorded under the item_quantity field of the Listing model, ListingItem's purpose is to show that each individual item of a listing can be unique. This is to cater for future updates where users can sell similar items of different conditions under one listing. For example, a user can sell similar chairs of different qualities such as "Well used", "Used", "Brand new" under the same listing.

The category model is shared by both the Shop model and Listing model so that sorting by categories remains consistent across both the marketplace and stores.

4.3.4 Cart ERD

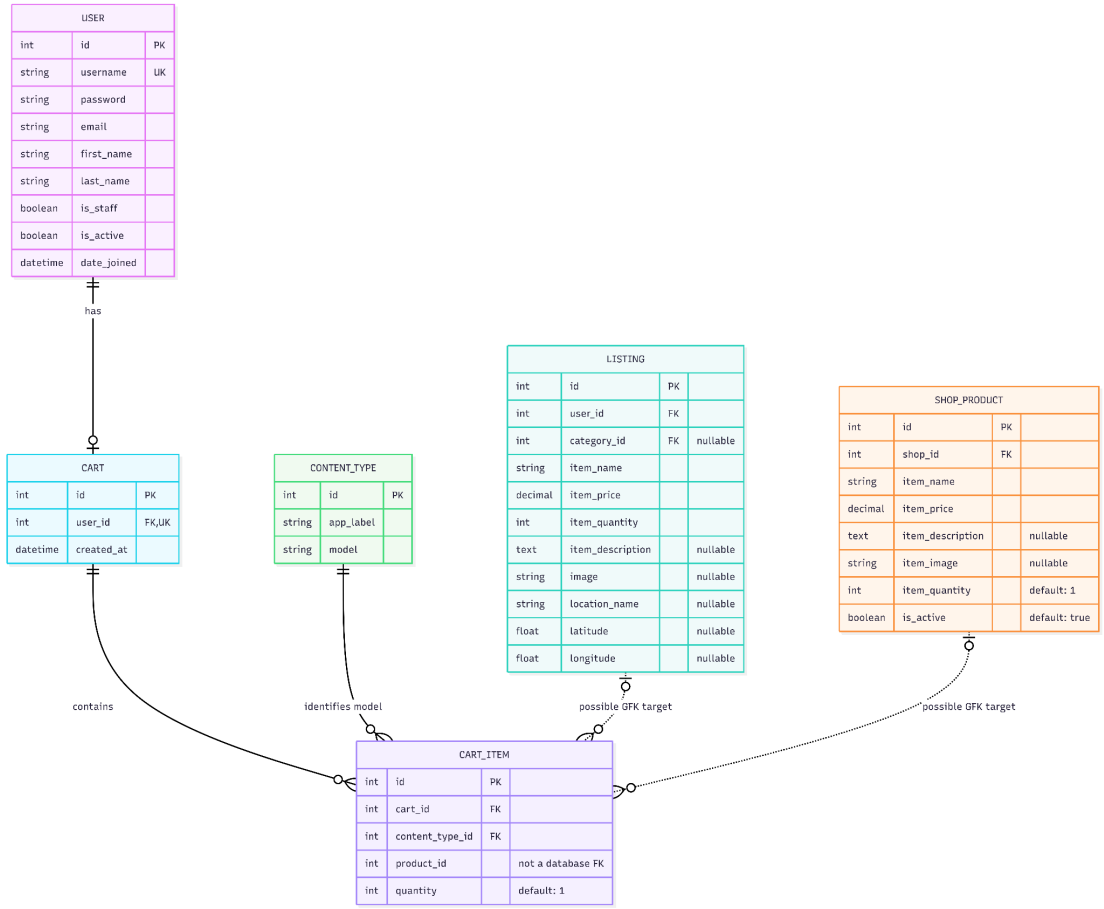


Figure 14. Cart ERD (Cart ERD.png)

As seen from Figure 14, every user may own one unique and persistent cart which can contain multiple Cart_Item rows. Each Cart_Item can reference either a Listing or a Shop_Product using Django's generic foreign-key column. This relationship is indicated by dotted lines in the ERD to show that it is purely conceptual, and that product_id is not a database enforced foreign key to either product tables.

The Content_Type model identifies which Django model each Cart_Item is representing. This allows the Cart_item to correctly identify which product the product_id that is saved is referencing. This allows Django to determine whether a stored value such as product_id = 5 refers to Listing 5 or Shop_Product 5.

4.3.5 Conversation ERD

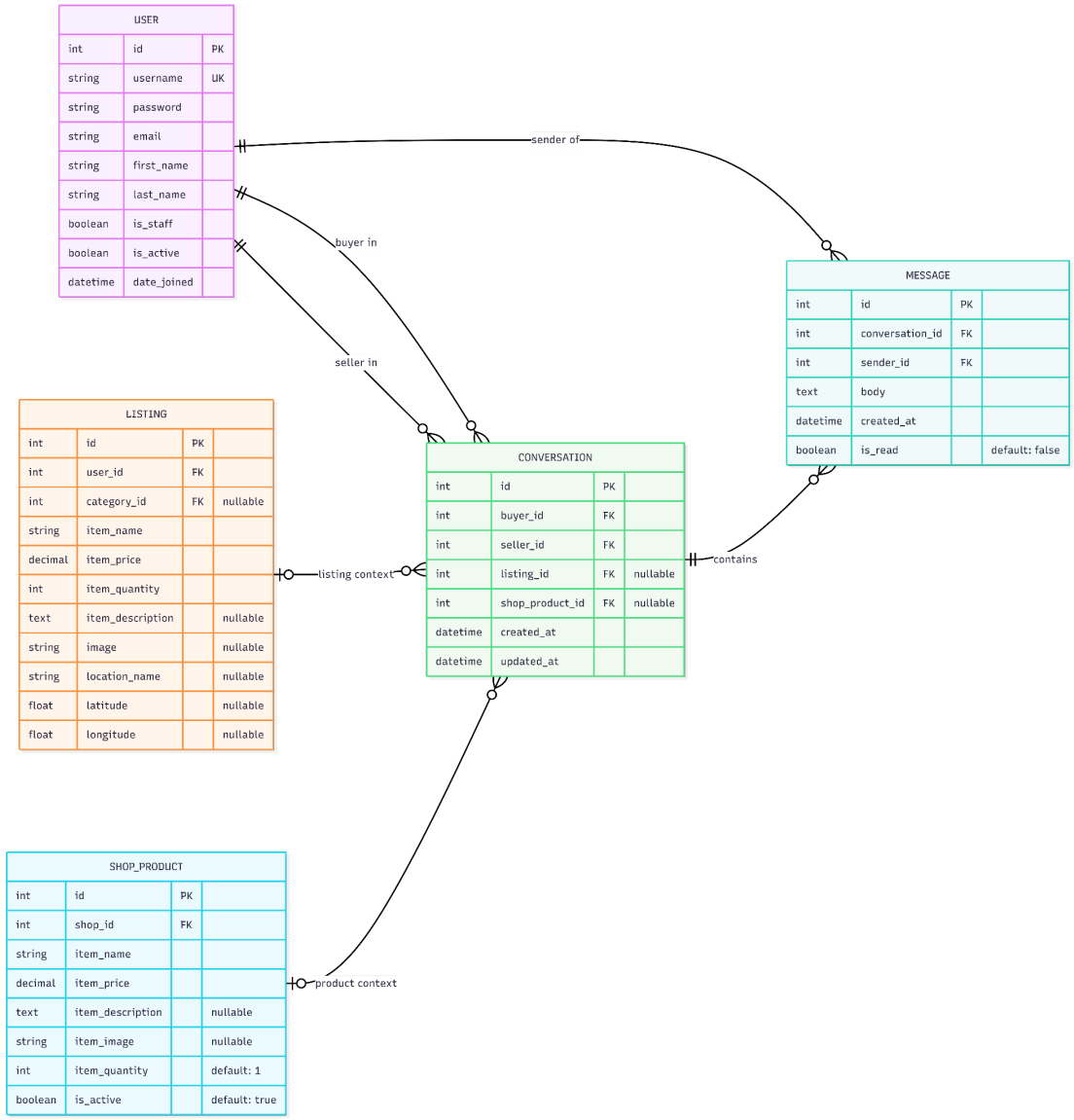


Figure 15. Conversation ERD (Conversation ERD.png)

As seen from Figure 15, the Conversation model represents a thread between a buyer and seller and stores both participants through foreign keys to the User model. Each conversation can contain multiple rows in the Message model, which makes up the chat.

The Conversation model also references the Listing and Shop_Product models which serves as context in the conversation. By including context in the conversation, each buyer/seller can have separate chats with the same user if the context is different.

4.4 Design Decisions

Decision	Justification
Separate Open Market and Stores	One-off sellers and repeat sellers have different needs. Separating them keeps casual resale simple while leaving room for richer shop management.
ListingItem rows for each unit	Unit-level inventory allows pending/sold status tracking and safer checkout than a single quantity number.
Generic content types in CartItem	A cart can eventually hold both marketplace listings and shop products without duplicating cart tables.
JWT authentication	A decoupled React frontend can authenticate against a Django backend without server-rendered sessions.
Cloudinary-ready ImageFields	Marketplace products need images; production deployment should not rely on local disk media.

5. Feature Breakdown

5.1 Feature 1: Dynamic Marketplace

Feature overview

The Dynamic Marketplace is the first core page of NUSphere. It allows students to post one-off items without needing to create a full store. This mirrors the way students currently sell items through Telegram channels and Carousell, but improves organization by storing each listing in a database and rendering it through a searchable, category-aware interface.

The marketplace is especially important for hostel move-out periods. Seniors can declutter items such as keyboards, chairs, appliances, clothing, and dorm essentials, while juniors can purchase useful items at lower prices. Unlike a Telegram group, the marketplace gives each item structured fields: name, price, quantity, category, description, and image.

User problem and design justification

The design is intentionally low-friction. A seller with a single item should not be forced through a store setup flow, because that would add unnecessary overhead. Separating one-off marketplace listings from personal stores also keeps the product mental model clear, where Open Market is for casual resale, while Stores are for repeat sellers.

Implementation and frontend/backend/database link

On the frontend, marketplace browsing is handled through OpenMarket/Listings.jsx as shown in Figure 7, while listing creation uses OpenMarket/AddListing.jsx. The route structure is declared in Defaults/Navigation.jsx, with Open Market as the default route.

On the backend, /api/listings/ is mapped to AddListingView. A GET request returns available listings with inventory counts. A POST request creates a Listing and bulk-creates one ListingItem row for each available quantity. The relevant models are Categories, Listing, and ListingItem. The image is handled through Django ImageField, using local media during development or Cloudinary when production credentials are present.

Current progress

The app currently supports listing creation and listing browsing. Authenticated users can add listings, and the Open Market feed can display available items.

5.2 Feature 2: Personalized Stores

Feature overview

Personalized Stores are the second core marketplace surface. They are intended for repeat sellers who need a more stable storefront than a one-off listing. Examples include students selling baked goods, personalized beverages, 3D printing services, keyboard mods, tutoring sessions, or recurring dorm supplies.

This feature was inspired by e-commerce platforms such as Shopee and Grab, as well as student-run Telegram shops. The difference is that NUSphere keeps the audience hyper-local to NUS, which reduces marketing overhead for student sellers.

User problem and design justification

The design separates seller modes. A student selling one used item should use Open Market; a student repeatedly selling products or services should use a Store. This distinction supports both simplicity and scalability. It also helps buyers understand whether they are looking at a one-time listing or a seller with an ongoing catalogue.

Implementation and frontend/backend/database link

On the frontend, store-related pages include StoresPage.jsx (Figure 9), AddStore.jsx, SelectMyStore.jsx, MyStore.jsx, and AddProduct.jsx. These routes let users create stores, select their stores, view products, and add products to a store.

On the backend, /api/store/ maps to AddStoreView, /api/store/personal/ maps to PersonalStoresView, and /api/store/storeitems/<store_id> maps to StoreItemView. The database models are Shop, ShopProduct, and ShopOrder. Shop is tied to a User through a one-to-one owner relationship, while ShopProduct belongs to a Shop.

Current progress

Currently implemented. Authenticated users can create stores and put up store products and all users can visit stores and view their listed products.

5.3 Feature 3: Map integration

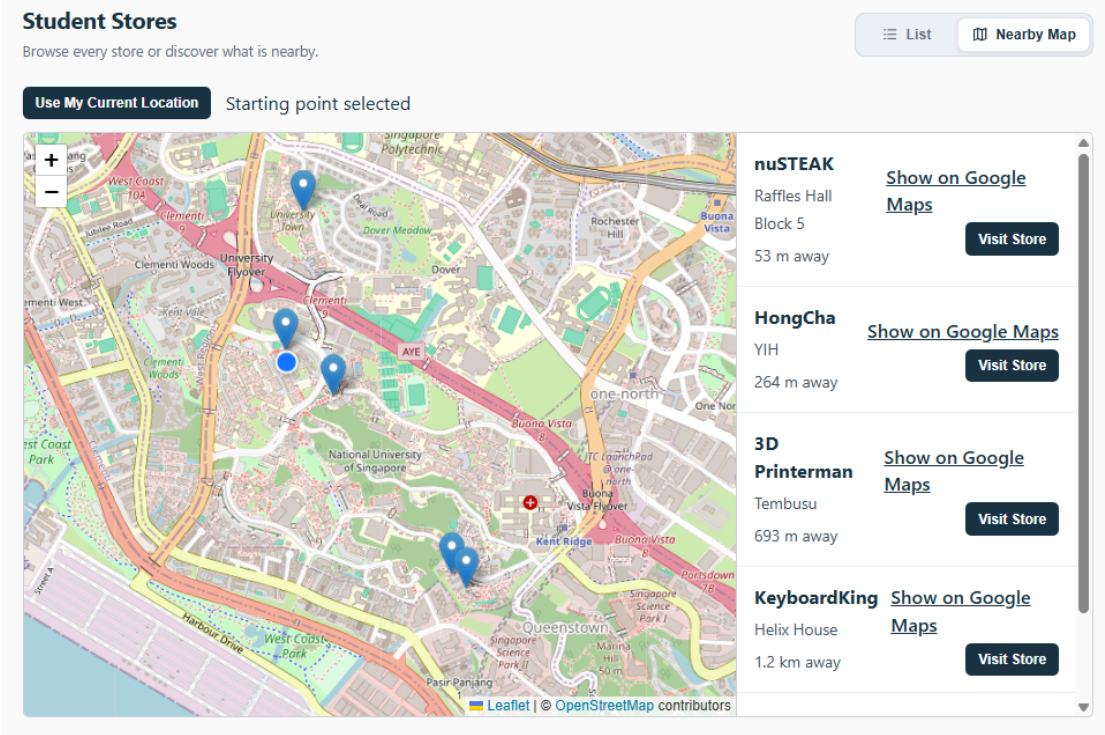


Figure 16. Example of stores map

Feature overview

Maps allow sellers to provide a readable location name and select coordinates using an interactive map, allowing buyers to view the location or open it in Google Maps to navigate to meet-up spots.

Store locations are also displayed on a Nearby Stores map, allowing buyers to locate nearby stores using their current location, or by selecting a specific location.

User problem and design justification

Campus transactions usually involve physical collection. Including just a location name may be ambiguous, especially for users who may be new to the school, or have not been to that part of campus. Using latitude and longitude to provide an exact geographical position, while offering a location name allows users to easily navigate to their meet-up spots or to find a store.

The interface allows users to click the map, drag the marker or use the browser's current-location feature so that users do not need to manually input coordinates.

Implementation and frontend/backend/database link

On the frontend, LocationPicker.jsx uses React Leaflet and Leaflet to display OpenStreetMap tiles. It is used by AddListing.jsx, AddStore.jsx and store-management pages. The Browser

Geolocation API can obtain the user's current coordinates, while map click and marker-drag events allow manual selection of locations. NearbyStoresMap.jsx displays stores with valid coordinates. Store and listing interfaces use buildGoogleMapsUrl() to generate an external Google Maps link from the saved latitude and longitude.

On the backend, AddListingView and AddStoreView validate and save location_name, latitude and longitude. Latitude must be between -90 and 90, while longitude must be between -180 and 180. Locations are optional for Open Market listings but required when creating a store.

The Listing and Shop database models store the location name and coordinates. The database stores coordinates rather than map images or directions; Leaflet, OpenStreetMap and Google Maps interpret those coordinates on the frontend.

Current progress

Currently implemented. Sellers can select locations for listings and stores without manually entering coordinates. Buyers can view stored locations, open them through Google Maps and locate stores through the Nearby Stores map.

5.4 Feature 4: Filtering and Sorting

Feature overview

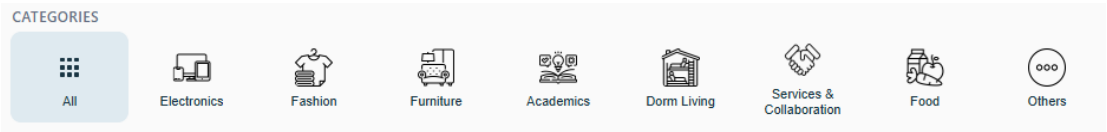


Figure 17. Example of category bar.

We implemented filtering in the main Marketplace and Stores tabs to help buyers reduce the number of products or stores that they need to browse through.

Category filtering and searching for relevant keywords is available for both the Marketplace listings and personalised stores.

Exclusive to stores, the nearby map feature allows users to browse through stores based on proximity by selecting a starting location or their current location.

User problem and design justification

As more products and stores are added, presenting every result in one unfiltered list becomes difficult for users to find what they are interested in. Categories allow users to narrow results according to their current needs, and distance sorting addresses the physical nature of campus transactions by prioritising stores that are closer to the users.

The two mechanisms can be applied together, where the map will only show nearby stores of the specified category, or nearby stores with the relevant keywords searched in the search bar.

Implementation and frontend/backend/database link

On the frontend, the selected category and search term are shared with Listings.jsx and StoresPage.jsx through the React Router outlet context. The components compare each listing or store category against the selected category. Selecting “All” leaves the complete collection visible.

For distance sorting, NearbyStoresMap.jsx accepts the filtered stores and a buyer location. The buyer can use the Browser Geolocation API or click the map to choose a starting point. The component applies the Haversine formula to calculate the approximate straight-line distance between the buyer and each store. Stores are then sorted from the smallest to largest calculated distance.

Categories are represented by the Categories database model and referenced by both Listing and Shop. Location coordinates are stored in the Shop model. Filtering and distance calculations currently occur in the React frontend after data has been retrieved from the backend.

Current progress

Currently implemented. Category filtering works for Marketplace listings and stores. Distance sorting works for stores with saved coordinates in the Nearby Stores view. If the buyer has not selected a location, stores are displayed alphabetically instead. Open Market listings currently only provide location information and Google Maps links, but are not sorted by distance.

5.5 Feature 5: Persistent Shopping Cart, Quantity Sync, and Shared Cart

Feature overview

The Persistent Shopping Cart allows a buyer to collect items before checkout. This is inspired by established e-commerce platforms, where students may browse both one-off marketplace listings and student-run stores, then coordinate purchasing decisions in one place.

A cart also improves user experience because buyers do not need to immediately commit while browsing. They can compare options, adjust quantity, and return to the cart later in the same authenticated session. The cart is not a static list; buyers can change item quantities from the frontend and those changes are synchronized back to the backend cart state.

User problem and design justification

The cart is designed as a user-bound resource rather than a temporary local state. This is important because cart contents should persist beyond a single component render. It also prevents carts from being lost when users navigate between Open Market, Stores, and profile/order pages.

The current design uses Django content types in CartItem. This is a deliberate design choice so that the same cart can eventually support different product-like models, such as marketplace Listing records and store ShopProduct records, without creating separate cart tables for each product type. This future direction matters because buyers should eventually be able to add store listings and store products to the same cart as Open Market items.

Implementation and frontend/backend/database link

On the frontend, cart behavior is grouped around UserSpecifics/CartContext.jsx, CartItems.jsx, SelectQuantityCart.jsx, and CartButton.jsx. CartContext fetches cart contents, adds or changes quantities, deletes items, and coordinates checkout calls. Quantity changes are sent to the Django cart endpoint, so the visible quantity is intended to stay aligned with the persisted CartItem quantity.

On the backend, /api/cart/ maps to CartAPIView. GET creates or returns the current user's Cart and serializes its items. POST adds to an existing item or changes quantity depending on post_type. DELETE removes a cart item. The database models are Cart and CartItem. Cart has a one-to-one relationship with User to ensure that each user only has one cart, while CartItem uses content_type and product_id to reference a product.

Current progress

Fully implemented, allowing users to add both Marketplace listings and store items to cart.

5.6 Feature 6: Real-Time Chat

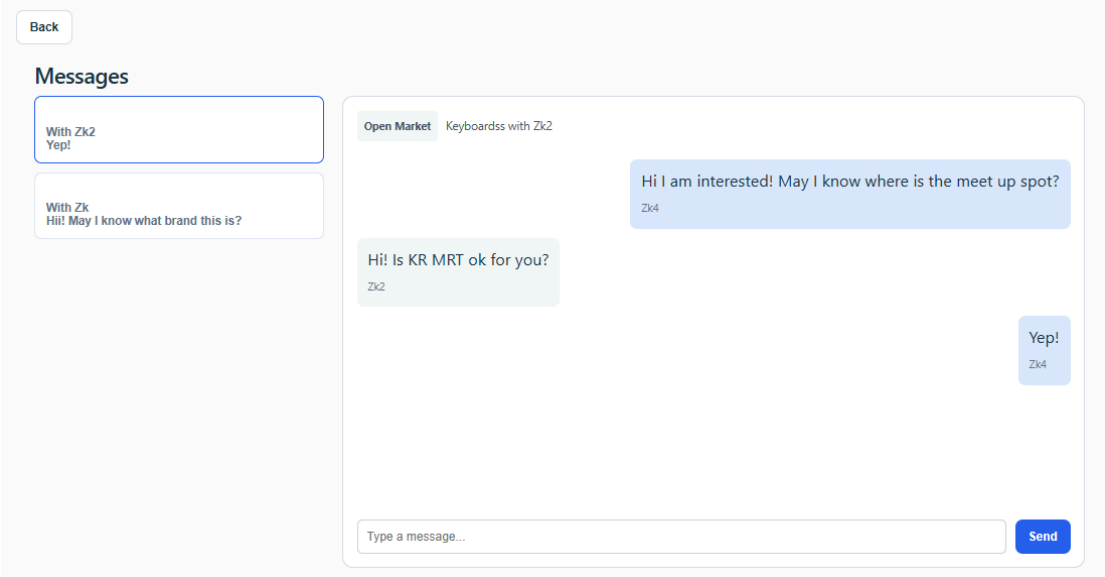


Figure 18. Example of chat function.

Feature overview

Real-time chat provides a secure, instantaneous communication layer directly within the platform. It allows buyers and sellers to discuss item details, negotiate meetup locations, and resolve transaction questions seamlessly. This feature mirrors the conversational flow found in platforms like Carousell chat and Facebook Marketplace.

User problem and design justification

The core design problem is transaction coordination and user privacy. Campus exchanges inherently require back-and-forth negotiation regarding meetup times and pickup points. If this communication moves outside the platform, the system loses visibility into user interactions, and users are forced to expose personal contact information (like phone numbers or personal social media handles) to strangers. Keeping the conversation internal protects user privacy and retains engagement.

Implementation and frontend/backend/database link

On the frontend, marketplace listings and store products contain a message seller button. Selecting it creates or retrieves the relevant conversation and navigates the user to Messages.jsx This component displays the users' conversations, retrieves messages for the selected conversations and allows users to send new messages to each other.

On the backend, /api/chat/conversations/ maps to ConversationListView, while /api/chat/conversations/<conversation_id>/messages/ maps to ConversationMessagesView. The backend verifies that the authenticated user is either the buyer or seller before returning a conversation or accepting a message.

The main database models are Conversation and Message. Conversation stores the buyer, seller and optional product context. Message stores its conversation, sender, messages and read status.

Current progress

Currently implemented. Authenticated users can start conversations from OpenMarket listings and store products. Messages are refreshed through three-second HTTP polling.

5.7 Feature 7: Order History and Pending Order Management

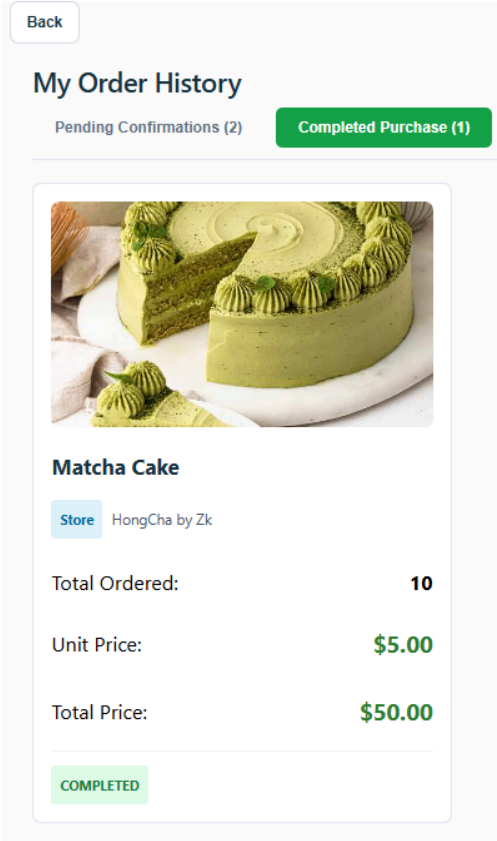


Figure 19. Example of completed purchases

Feature overview

Order history and pending order management form the post-checkout workflow. After checkout, the buyer needs a place to see what has been requested, while the seller needs a page that shows which pending purchases require action. This makes the feature more than a static history page, it coordinates two user perspectives after a transaction begins.

The buyer-facing order history shows pending and sold purchases, so students can distinguish items still awaiting seller confirmation from items that have already been confirmed. The seller-facing pending-order page acts like a fulfilment queue for marketplace sellers, allowing them to confirm orders when they have the capacity to deliver.

User problem and design justification

This feature matters because campus transactions often end in real-world handoff. A buyer may check out first, then meet the seller later. The system therefore needs an intermediate pending state instead of immediately treating checkout as final completion. This distinction supports the future escrow and QR workflow, where order states could move from pending to completed only after verification.

Implementation and frontend/backend/database link

On the frontend, OpenMarket/MyOrders.jsx retrieves buyer order history, while OpenMarket/PendingListing.jsx lets sellers view pending purchases for their own listings and confirm them.

On the backend, /api/listings/orderhistory/ maps to OrderHistoryView and separates pending and sold purchases for buyers. /api/listings/confirmsold/ maps to ConfirmSoldView and handles seller-side confirmation.

The database models involved are Order, ListingItem, Listing, Shop, ShopProduct, and ShopOrder. Order and ShopOrder connect a buyer to purchased inventory, while ListingItem and ShopOrder statuses track whether each item is pending or sold.

Current progress

Currently implemented. Users are now able to see their order history and orders that are pending seller’s confirmation. Users are also able to view orders from buyers that are pending their confirmation.

5.8 Feature 8: Seller Analytics Dashboard

Feature overview

The Seller Analytics Dashboard is planned as a store-owner and seller-management feature. Its purpose is to help student entrepreneurs understand business performance: revenue, pending orders, completed orders, stock counts, low-stock products, and best-selling items.

This feature is inspired by Shopify Analytics and creator dashboards, but scaled down for student sellers. A student running a small food or service business should be able to see which products are moving and when to restock.

User problem and design justification

The design justification is that repeat sellers need feedback loops. Without analytics, sellers must manually infer demand from chats or memory. A dashboard reduces that burden by converting order and inventory data into actionable summaries.

The dashboard also supports the project's entrepreneurship angle. NUSphere is not only a resale board; it is a platform that can help students practice small-business operations in a campus-safe environment.

Implementation and frontend/backend/database link

On the frontend, UserSpecifics/SellerAnalytics.jsx retrieves information such as total revenue, pending orders, completed orders, inventory snapshot and best-selling items to assist sellers on making future business decisions.

On the backend, api/seller/analytics maps to SellerAnalyticsView which returns a user’s inventory health, sale history and relevant business information.

Current progress

Currently implemented, users can now see their personalized dashboard under their profile tab.

5.9 Feature 9: Checkout and Inventory Reservation

Feature overview

Checkout and inventory reservation convert cart intent into committed orders. This feature ensures that when users check out from their cart, the item is reserved and other users will not be able to check out the same item, causing duplicate orders.

User problem and design justification

The design reflects a key difference between a campus marketplace and a simple static catalogue. Listings may have limited stock, and multiple buyers may attempt to buy the same item around the same time. A checkout design that only trusts frontend quantity would be unsafe because the browser could have stale data.

For this reason, the backend performs the authoritative stock check at checkout time. The frontend cart is useful for planning, but the database is the source of truth for inventory. Marketplace inventory is represented through individual ListingItem rows, allowing each physical unit to move from their initial “unsold” state, to pending and sold states. Unlike Marketplace inventory which accounts for the uniqueness of each unit of item per listing, Store inventory is represented through a simple numerical item_quantity field to track how many items are in stock.

Implementation and frontend/backend/database link

On the frontend, UserSpecifics/CartContext.jsx calls /api/listings/checkout/ after the buyer confirms checkout from the cart. The cart page supplies the selected products and quantities, while the backend validates those values before mutating database states.

On the backend, CheckOutView handles availability checks and order creation for marketplace listings checkouts, while StoreCheckOut handles store product checkouts. The backend ensures that no duplicate orders are created by locking the requested number of unsold ListingItem rows or ShopProduct rows respectively and checking if the required quantity is available before creating their respective orders. In the event the required quantity is not available, an error will be thrown, and no changes to the database will be made.

The database models involved are Listing, ListingItem, Order, Shop, ShopProduct and ShopOrder.

Current progress

Currently implemented. Users are now able to check out both Marketplace items and store items successfully.

6. Tech Stack

React and Vite provide fast frontend development and reusable UI components.

Django REST Framework provides structured backend APIs, authentication integration, ORM models, and database migrations.

PostgreSQL supports relational data that is central to the project: users, listings, shops, carts, and orders.

Layer	Technology	Reason for use
Frontend	React, Vite, React Router	Reusable components, fast development server, and route-based product surfaces.
API client	Axios and fetch	HTTP requests from components and contexts to Django endpoints.
Backend	Django, Django REST Framework	Mature ORM, authentication, APIView routing, serializers, and admin/migration support.
Authentication	SimpleJWT	Token-based authentication for a decoupled frontend and backend.
Database	PostgreSQL	Relational persistence for users, inventory, orders, carts, stores, and products.
Media	Django ImageField, local media, Cloudinary-ready config	Product, listing, store, and profile images need persistent storage.
Deployment	Railway-oriented settings, WhiteNoise, CORS headers	Supports deployed proof of concept and controlled frontend/backend communication.

7. Testing Strategy

7.1 Automated tests:

7.1.1 Backend tests:

Our backend automated tests use Django REST Framework's APITestCase to test the URL routing, authentication, API views, Django ORM operations and database state changes. Cases such as positive, negative, boundary-value and authorization cases were used in the tests.

```
test_listing_can_be_added_to_cart (cart.tests.CartTests.test_listing_can_be_added_to_cart) ... ok
test_login_is_required_to_view_cart (cart.tests.CartTests.test_login_is_required_to_view_cart) ... ok
test_product_from_closed_store_cannot_be_added (cart.tests.CartTests.test_product_from_closed_store_cannot_be_added) ... ok
test_product_from_open_store_can_be_added (cart.tests.CartTests.test_product_from_open_store_can_be_added) ... ok
test_buyer_can_start_conversation_about_listing (chat.tests.MessagingTests.test_buyer_can_start_conversation_about_listing) .
.. ok
test_empty_message_is_rejected (chat.tests.MessagingTests.test_empty_message_is_rejected) ... ok
test_other_user_cannot_read_conversation (chat.tests.MessagingTests.test_other_user_cannot_read_conversation) ... ok
test_listing_location_requires_name_and_both_coordinates (listings.tests.ListingLocationTests.test_listing_location_requires_
name_and_both_coordinates) ... ok
test_listing_rejects_coordinates_outside_valid_range (listings.tests.ListingLocationTests.test_listing_rejects_coordinates_ou
tside_valid_range) ... ok
test_listing_saves_valid_location_and_inventory_units (listings.tests.ListingLocationTests.test_listing_saves_valid_location_
and_inventory_units) ... ok
test_checkout_accepts_exact_available_quantity (listings.tests.MarketplaceCheckoutTests.test_checkout_accepts_exact_available
_quantity) ... ok
test_checkout_rejects_quantity_above_available_stock (listings.tests.MarketplaceCheckoutTests.test_checkout_rejects_quantity_
above_available_stock) ... ok
test_checkout_rejects_zero_quantity (listings.tests.MarketplaceCheckoutTests.test_checkout_rejects_zero_quantity) ... ok
test_listing_owner_can_confirm_pending_order (listings.tests.MarketplaceConfirmationTests.test_listing_owner_can_confirm_pend
ing_order) ... ok
test_other_user_cannot_confirm_pending_order (listings.tests.MarketplaceConfirmationTests.test_other_user_cannot_confirm_pend
ing_order) ... ok
test_order_history_combines_marketplace_and_store_orders (listings.tests.OrderHistoryTests.test_order_history_combines_market
place_and_store_orders) ... ok
test_login_rejects_wrong_password (login.tests.LoginTests.test_login_rejects_wrong_password) ... ok
test_valid_login_returns_access_and_refresh_tokens (login.tests.LoginTests.test_valid_login_returns_access_and_refresh_tokens
) ... ok
test_login_is_required_to_view_profile (login.tests.ProfileTests.test_login_is_required_to_view_profile) ... ok
test_user_can_change_username (login.tests.ProfileTests.test_user_can_change_username) ... ok
test_username_must_be_unique (login.tests.ProfileTests.test_username_must_be_unique) ... ok
test_registration_rejects_different_passwords (login.tests.RegistrationTests.test_registration_rejects_different_passwords) .
.. ok
test_user_can_register_with_matching_passwords (login.tests.RegistrationTests.test_user_can_register_with_matching_passwords)
 ... ok
test_open_status_must_be_boolean (shop.tests.PersonalStoreSettingsTests.test_open_status_must_be_boolean) ... ok
test_store_location_update_requires_all_location_fields (shop.tests.PersonalStoreSettingsTests.test_store_location_update_req
uires_all_location_fields) ... ok
test_authenticated_seller_can_view_their_analytics (shop.tests.SellerAnalyticsIntegrationTest.test_authenticated_seller_can_v
iew_their_analytics) ... ok
test_login_is_required_to_view_analytics (shop.tests.SellerAnalyticsIntegrationTest.test_login_is_required_to_view_analytics)
 ... ok
test_checkout_reduces_store_stock (shop.tests.StoreCheckoutIntegrationTest.test_checkout_reduces_store_stock) ... ok
test_closed_store_rejects_checkout (shop.tests.StoreCheckoutIntegrationTest.test_closed_store_rejects_checkout) ... ok
test_other_user_cannot_confirm_store_order (shop.tests.StoreConfirmationTests.test_other_user_cannot_confirm_store_order) ...
 ok
test_store_owner_can_confirm_pending_order (shop.tests.StoreConfirmationTests.test_store_owner_can_confirm_pending_order) ...
 ok

----------------------------------------------------------------------
Ran 32 tests in 0.190s

OK
```

Figure 20. CLI of backend tests

Index	Scenario	Technique	Expected and actual result
1	Analytics without login	Negative	401 Unauthorized, passed

2	Authenticated seller analytics	Positive	Correct store, inventory and low-stock totals, passed
3	Checkout from closed store	Negative	400 Bad Request; stock and orders unchanged, passed
4	Checkout exact store stock	Boundary/positive	Order created and stock becomes zero, passed
5	Start listing conversation	Positive	Conversation created with correct buyer, seller and listing, passed
6	Send empty message	Negative	400 Bad Request; no message created, passed
7	Outsider reads conversation	Authorization	404 Not Found, passed
8	Marketplace checkout quantity zero	Boundary/negative	Rejected without creating orders, passed
9	Checkout exact marketplace stock	Boundary/positive	Two orders created, and two units reserved, passed
10	Checkout above marketplace stock	Boundary/negative	Rejected without inventory changes, passed
11	Listing owner confirms order	Positive	ListingItem becomes sold and order becomes confirmed, passed
12	Another user confirms listing order	Authorization	404 Not Found; statuses unchanged, passed
13	Combined order history	Integration/positive	Marketplace and store orders returned together, passed
14	Register with matching passwords	Positive	201 Created; user created with hashed password, passed

15	Register with different passwords	Negative	400 Bad Request; user not created, passed
16	Login with valid credentials	Positive	Access and refresh JWT tokens returned, passed
17	Login with incorrect password	Negative	401 Unauthorized, passed
18	View profile without login	Authorization	401 Unauthorized, passed
19	Change username	Positive	Username updated in response and database, passed
20	Change username to existing username	Negative	400 Bad Request; original username retained, passed
21	View cart without login	Authorization	401 Unauthorized, passed
22	Add marketplace listing to cart	Positive	Cart item created with correct listing and quantity, passed
23	Add zero quantity to cart	Boundary/negative	400 Bad Request; no cart item created, passed
24	Add product from closed store	Negative	400 Bad Request; no cart item created, passed
25	Add product from open store	Positive	Cart item created and generic foreign key resolves ShopProduct, passed
26	Listing location missing coordinates	Negative	400 Bad Request; listing not created, passed
27	Listing latitude outside valid range	Boundary/negative	400 Bad Request; listing not created, passed

28	Listing with valid location and quantity	Positive	Location saved and two ListingItem rows created, passed
29	Store owner confirms pending order	Positive	Store order status becomes sold, passed
30	Another user confirms store order	Authorization	404 Not Found; order remains pending, passed
31	Store open status supplied as text	Negative	400 Bad Request; store remains closed, passed
32	Store location update missing fields	Negative	400 Bad Request; location remains unchanged, passed

7.1.2 Frontend tests:

We conducted our frontend tests using Vitest and React Testing Library.

Unit tests were used to verify the Google Maps and Haversine utility functions using controlled inputs and expected outputs.

Component tests were used to render individual components in JSDOM and inspect their visible output and user interactions

Lastly, network requests were mocked so frontend behaviour can be tested independently of the deployed backend.

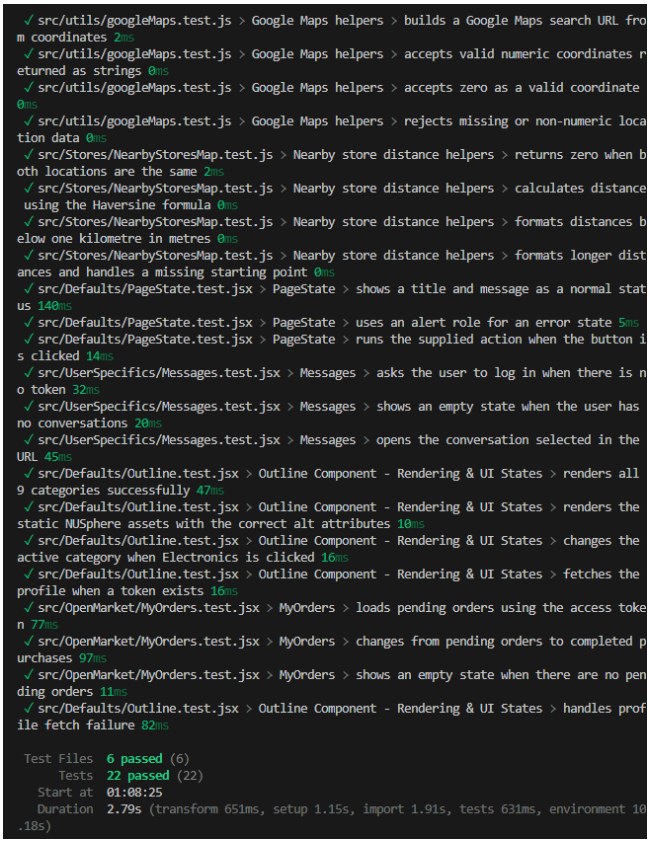


Figure 21. CLI of frontend tests

Index	Scenario	Technique	Expected and actual result
1	Render all marketplace categories	Positive	All nine categories displayed, passed

2	Render NUSphere assets	Positive	Logo and name image displayed with correct attributes, passed
3	Select Electronics category	Interaction/positive	Electronics becomes active and All becomes inactive, passed
4	Fetch profile with access token	Integration/positive	Request includes the correct bearer token, passed
5	Profile request fails	Negative	Failure is handled and logged without crashing, passed
6	Render normal page state	Positive	Status role, title and message displayed, passed
7	Render error page state	Negative	Alert role and error styling applied, passed
8	Click page-state action	Interaction	Supplied retry function called once, passed
9	Build Google Maps URL	Unit/positive	Correct URL produced from latitude and longitude, passed
10	Validate numeric coordinate strings	Unit/positive	Location accepted as valid, passed
11	Validate zero coordinates	Boundary/positive	Latitude and longitude of zero accepted, passed
12	Validate missing or non-numeric coordinates	Unit/negative	Invalid locations rejected, passed
13	Calculate distance between identical points	Boundary	Distance returned as zero, passed

14	Calculate one-degree Haversine distance	Unit/positive	Approximately 111.2 km returned, passed
15	Format distance below one kilometre	Boundary	0.45 km displayed as 450 m away, passed
16	Format long or unavailable distance	Unit/boundary	Kilometres formatted correctly and missing location prompt shown, passed
17	Load pending order history	Integration/positive	Authenticated request made and order totals displayed, passed
18	Change to completed purchases tab	Interaction/positive	Completed order displayed and pending order hidden, passed
19	Display empty pending-order state	Boundary	“No pending orders found” displayed, passed
20	Open messages without login	Authorization	Login prompt displayed and no API request made, passed
21	Load account with no conversations	Boundary	“No conversations yet” displayed, passed
22	Open conversation selected in URL	Integration/positive	Correct conversation and messages retrieved and displayed, passed

7.2 User Testing Plan

User testing should involve students who represent both buyers and sellers. Each participant would be given short tasks such as finding an item, adding it to cart, creating a listing, creating a store product, or explaining what pending orders mean.

Participant	Task	Completed	Time	Rating	Comments
Zeph Tang	Find pastries on the listings	Yes	2.4s	4/5	Easy to find through search bar

Patil Shlok	Create an account	Yes	13s	3/5	The login and registration page is quite plain and there are no restrictions on the account details like password strength
Lim Zheng Feng	Add own listing	No	24s	2.5/5	Couldn't find the tab for creating only listings, only found out after asking for help
Hur Wan	Add 3 items to cart	Yes	11s	4/5	Simple process for adding items to cart, quite intuitive
Hur Wan	Check out the 3 items	Yes	3.3s	5/5	Cart button was easy to locate, check out was hassle-free perhaps due to lack of messaging seller
Lee Ze Bin	Locate COM Corner Tech in Stores	Yes	38s	4.5/5	Took a while to figure out the "Stores" tab but afterwards, it was simple to locate the store
Jin Donghua	Set up a personal store and view seller analytics	No	58s	3/5	Difficulty finding the seller analytics page as it was hidden within the profile tab.

User testing was conducted with six students representing likely buyers and sellers. Participants completed seven tasks, covering product discovery, account creation, cart usage listing creation, store setup and seller analytics.

Metric	Result
Overall completion rate	71.4%
Median completion time	13 seconds
Average completion time	21.4 seconds
Average ease rating	3.7/5

Some key findings include: The Search function enabled fast product discovery, adding products to the cart was intuitive and the cart and checkout controls were easy to locate. The most significant problem was feature discoverability. Both unsuccessful tasks involved seller functions that participants could not locate without assistance.

Recommended improvements would be to add a clearly labelled Sell or Create Listing button to the main navigation and introduce a dedicated Seller Dashboard navigation item instead of hiding it inside the profile menu.

8. Timeline

MS	Tasks	Description	In-Charge	Date
1	Poster and video design	Design poster and video	Hong Meng	11 - 17 May
	Pick up necessary skills	Learn React, Django and PostgreSQL	Zhi Kai	18 - 24 May
	Develop Front-end skeleton	Draft what the front end will look like and create using react	Hong Meng	18 - 24 May
	Develop Backend-Frontend integration	Integrate front end react with backend Django and PostgreSql	Zhi Kai	18 -24 May
	Merge and create general idea of marketplace	Add listing capabilities of website for MarketPlace	Zhi Kai	25 - 31 May
Milestone 1 Submission - 1 June				
MS 2	Clean user interface	Design images of different buttons on the web such as store buttons, categories etc.	Hong Meng	1 June - 7 June
	Create user account feature	Add logging in features to the website	Zhi Kai	1 June - 7 June
	Complete Shops component	Create UI and backend integration with	Hong Meng	8 June - 14 June

		django for personal shops		
	Complete Live Chat and notifications	Create live chat and email notifications	Zhi Kai	15 June - 21 June
	Smart recommendations	Create recommendations based on location	Hong Meng	21 June - 28 June
Milestone 2 Submission - 29 June				
MS 3	Complete Escrow payment	Develop escrow payment and QR scanning upon successful deal	Hong Meng	29 June - 5 July
	Telegram chat Integrations	Integrate shop and marketplace notifications with telegram bot	Zhi Kai	6 July- 12 July
	Clean up UI and bug fixing	Clean up UI to scale with web size	Hong Meng	12 July - 26 July
		Fix any backend bugs	Zhi Kai	
Milestone 3 Submission - 27 July				

9. Proof of Concept and Project Links

- Proof of concept: https://nusphere-production.up.railway.app/open-market
- Project log: https://docs.google.com/spreadsheets/d/1xhzpKT1MyQlIDihrgsUQhM0fs6AHGvxgb9v7F1Hq5iU/edit?usp=sharing
- Repository layout:
    1. NUSphereFrontEnd contains the React/Vite application
    2. NUSphereBackend contains the Django project and apps.